UNIVERSITY OF RWANDA

College of Science and Technology

School of ICT

Department of Computer Engineering

Mobile Application Systems and Design

LAB 4: CONSUMING APIS IN FLUTTER

Posts Manager — Flutter Mobile Application

Submitted by:

Patrick Sraton Mbabazi

Date: March 16, 2026

1. Application Overview

The Posts Manager is a Flutter mobile application built for a small media company. It enables staff members to interact with a RESTful API (JSONPlaceholder) to perform full CRUD operations on posts. The application was developed as part of Lab 4 to demonstrate API integration in Flutter while focusing on the frontend experience.

The application features the following capabilities:

- View all available posts from the server
- Read the full detail of any individual post
- Create a new post via a form screen
- Edit an existing post with pre-filled data
- Delete a post from the list or detail view
- User authentication (Login / Logout flow)

API Endpoint used:

https://jsonplaceholder.typicode.com/posts

2. Discussion Questions (Handwritten)

The following pages contain the original handwritten answers to the discussion questions as submitted on 16/03/2026.

On 16/03/2026

University of Rwanda
College of Science and Technology
School of ICT
Department of Computer Engineering
Mobile Application Systems and Design

Lab 4 Consuming APIs in Flutter

Q1. Dependencies used

i. http: 1.2.0 : this is official Dart package for making HTTP request which provides simple cross platform ~~tiles to send~~ APIs to send GET, POST, PUT and DELETE request to JsonPlaceHolder API.

ii. cupertino_icons: 1.0.6 : provide iOS-style Cupertino icon assets included by default in Flutter projects for cross-platform icon consistency. used Alongside Material Icons in the UI.

iii flutter (SDK) + flutter_lints (dev) : the core Flutter SDK is always required. flutter_lints is a dev dependency providing lint rules that encourage good coding practices and help catch errors during development

Q2. Exception Handling Strategy

Our App uses a layered exception-handling approach defined in post_service.dart. A custom ApiException class wraps all errors types with a human readable message.

Types of Exaptions handled

- Socket Exception : Triggered when there is no internet connection or server is unreachable
- Format Exception : Triggered when the response body cannot be parsed as valid JSON (invalid data from server)
- Http Status Codes : 404 (not found), 201 (created) and other non-success codes are explicitly checked and mapped to meaningful errors
- Timeout Exception : A 30 second timeout is applied using .timeout(Duration(seconds: 15)) on every request
- Generic Catch : A final catch block catches any unexpected error and wraps it in ApiException to prevent crashes

In UI layer (FutureBuilder), when snapshot.hasError is true, the app will display a friendly error widget with a "Retry" button instead of crashing

Handwritten Discussion — Page 1 (Q1 & Q2)

Q3. Explain the Future Widget used and the UI parts that depend on it.

Future Builder Widget

FutureBuilder : Is a Flutter widget that builds itself based on the latest snapshot of interaction with a Future. It listens to the Future and automatically rebuilds the widget tree when the Future completes (either data or an error)

How FutureBuilder is used in our App

In PostsListScreen we declare: late Future _postsFuture; Here the future is assigned by calling _postsService.fetchPosts() and is passed directly to FutureBuilder.

The Three UI states driven by FutureBuilder:

- Connection State.waiting : Shows a centered green circular progressing indicator with the text "Tegereza kato...". The entire list area is replaced by this loader
- Snapshot.hasError == true : Shows an error icon, the error message (eg. Nta internet...), and retry button ("Ongera ugerageze"). The Entire list is hidden.
- Snapshot.hasData == true Renders the full ListView.builder with All posts as Cards, Each card shows the post title body preview, User ID and Action buttons

The same pattern is used in PostDetailScreen to fetch a single post : Future _postFuture controls whether the detail card loader or error state is shown

App summary

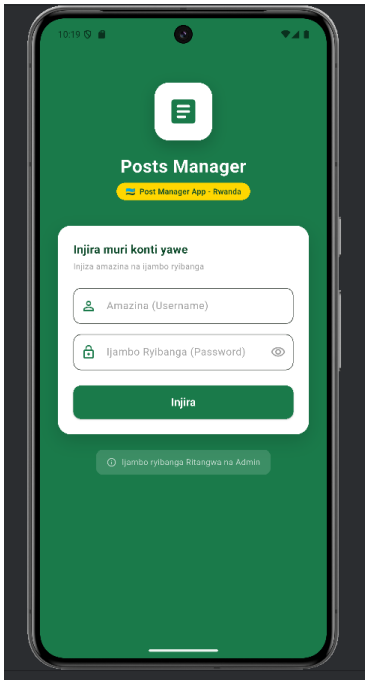
Future	Screen	HTTP Method	Endpoint
View All posts	PostsListScreen	GET	/posts
View Singlepost	PostDetailScreen	GET	/posts/{id}
Create post	PostFormScreen	POST	/posts
Edit post	PostFormScreen	PUT	/posts/{id}
Delete post	PostsListScreen/detail	DELETE	/posts/{id}

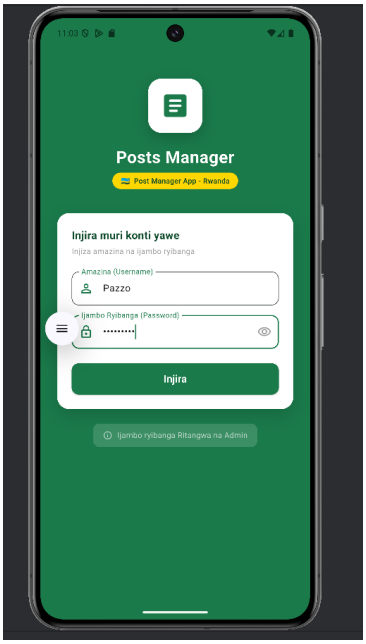
Handwritten Discussion — Page 2 (Q3 & App Summary Table)

3. Application Screenshots

The following screenshots demonstrate the full functionality of the Posts Manager Flutter application.

3.1 Login & Authentication





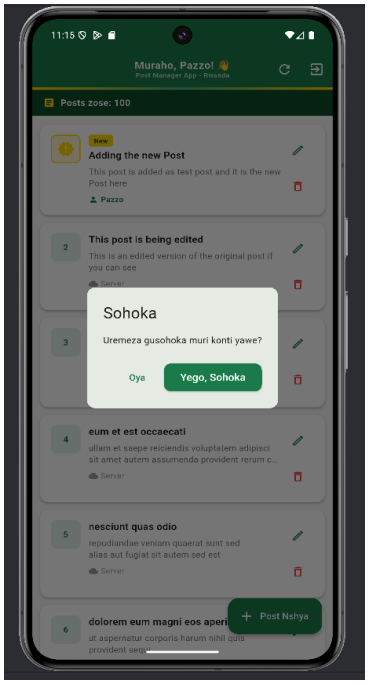
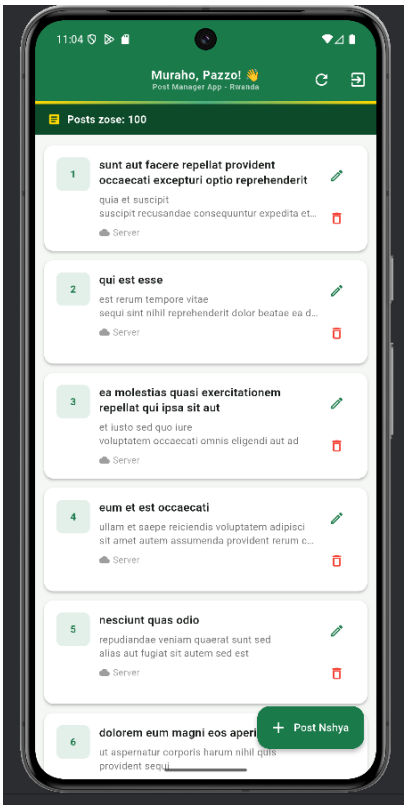


Figure 1: Login Page | Figure 2: Logging In | Figure 3: Logging Out

3.2 Viewing Posts



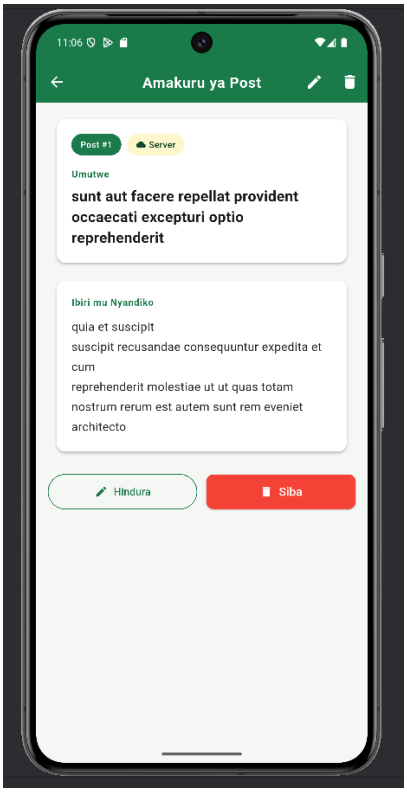
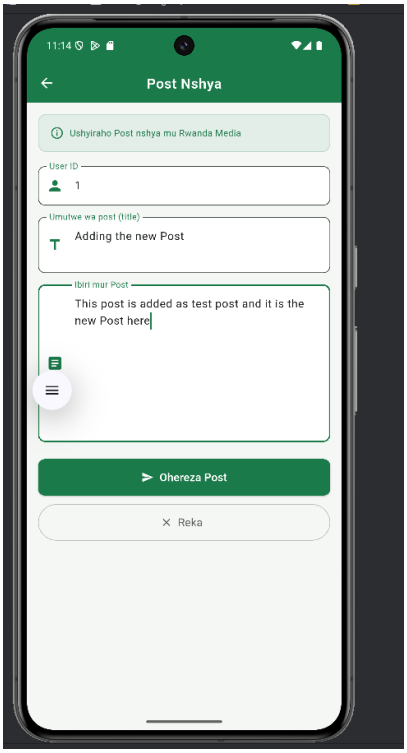


Figure 4: All Posts List | Figure 5: Post Detail View

3.3 Creating a Post



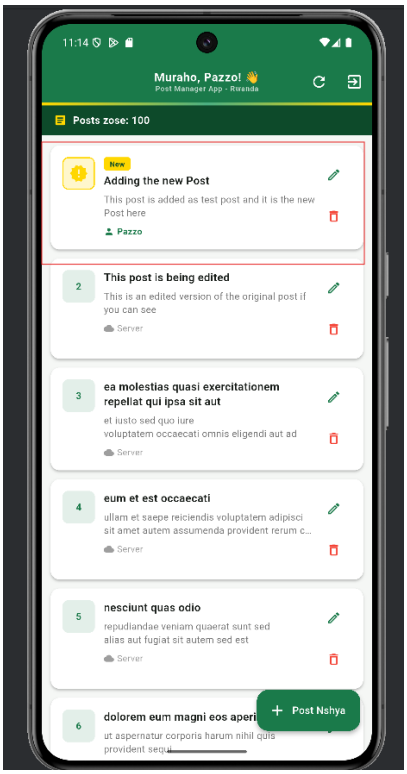
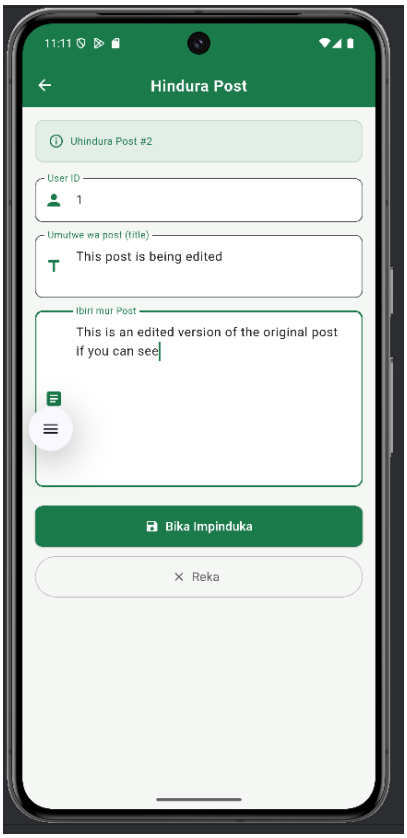


Figure 6: Creating a Post (Form) | Figure 7: Post Created Successfully

3.4 Editing a Post



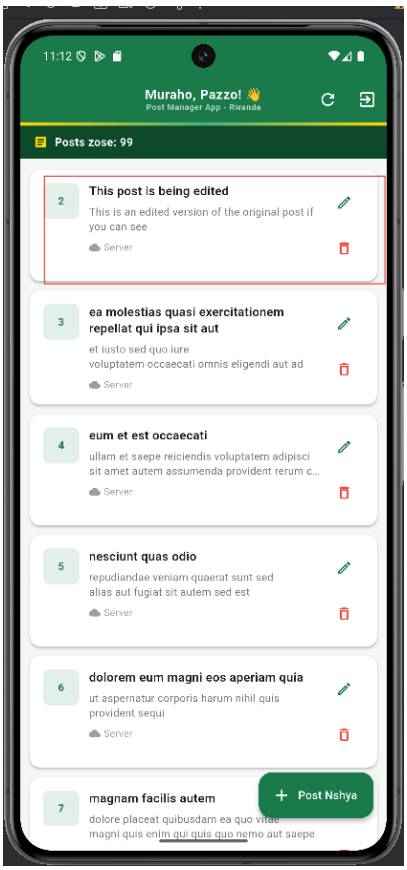
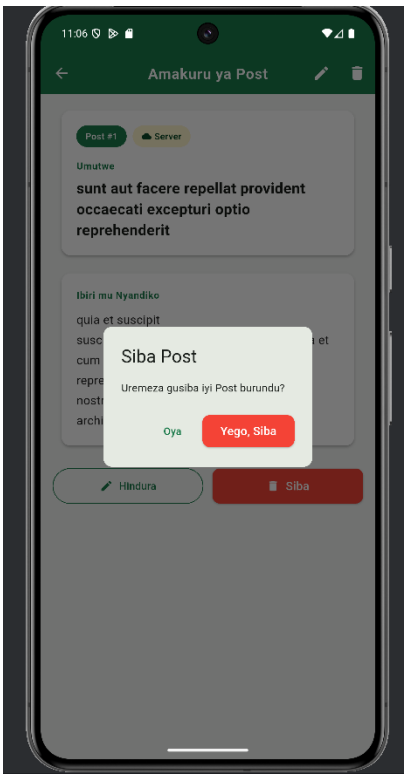


Figure 8: Editing a Post | Figure 9: Post Edited Successfully

3.5 Deleting a Post



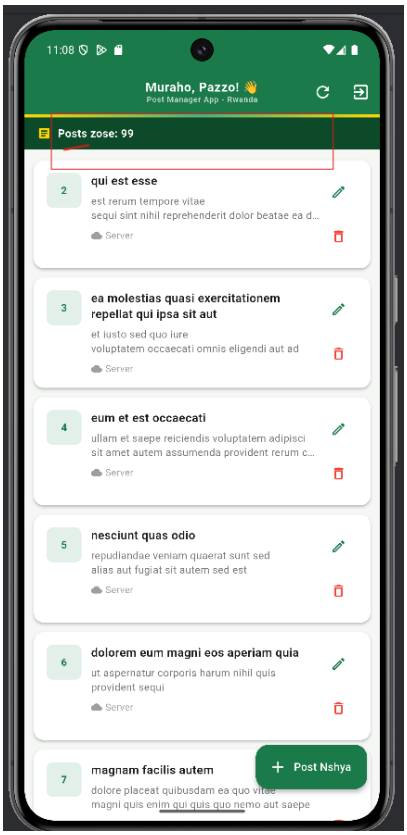


Figure 10: Deleting a Post (Confirmation) | *Figure 11: Post Deleted Successfully*